

Regiones de Operación Robustas en Sistemas Multidimensionales

Manual de Referencia en Python · v3 · Edición integrada con enfoque en trading algorítmico y series temporales

Código de color de los recuadros: **CRÍTICO** error que invalida el análisis · **INTEGRADO** ampliación nueva en esta versión · **TRADING** específico de series temporales / mercados.

TL;DR. Robusto = región estable (meseta), no pico óptimo. Flujo: (1) ajusta un sustituto sobre tus datos, (2) **válidalo de forma honesta — por regiones del espacio y, si hay tiempo, walk-forward, nunca k-fold aleatorio**, (3) sondéalo con Monte Carlo inyectando el ruido *físico real* (`sigma_eps` bien calibrado, no inventado), (4) puntúa cada candidato por media $\pm k \cdot \sigma$ y quédate con baja σ . Para sistemas con thresholds, stops o cambios de régimen, **abandona el GP suave** y usa árboles/quantile forests. En trading, el enemigo nº1 no es el ruido sino el **sobreajuste de backtest y la fuga temporal (lookahead)**. Visualiza con HiPlot/Plotly y contornos QbD; evita radar y t-SNE.

1. El núcleo: óptimo frágil vs. región robusta

Una región robusta no es un pico estrecho sino una **meseta estable** donde pequeñas perturbaciones ϵ alrededor de un candidato x^* dejan la salida $y(x^* + \epsilon)$ casi sin cambios. La formulación canónica es minimizar/optimizar $\mu \pm k \cdot \sigma$ (Beyer & Sendhoff, 2007), no `max f(x)` a secas.

TRADING · ESTO ES EXACTAMENTE EL PROBLEMA DEL OVERFITTING DE BACKTEST

El "pico estrecho" en trading tiene nombre: es el conjunto de parámetros que maximiza el backtest. Casi siempre es ruido, *lucky region* o artefacto histórico, y muere fuera de muestra. La "meseta robusta" — una zona amplia de parámetros con desempeño similar y aceptable — sobrevive mejor porque no depende de que el futuro replique un punto exacto del pasado. **Buscar la meseta es, literalmente, anti-overfitting.** Si tu superficie de Sharpe tiene un pico aislado rodeado de valles, desconfía; si tiene una zona ancha y suave, ahí hay señal real.

2. Visualización multidimensional (5–10 variables)

Coordenadas paralelas (HiPlot / Plotly) son la herramienta de cabecera: cada fila es una polilínea sobre un eje por variable. Pintando (*brushing*) el eje de salida a una banda estrecha, ves los rangos de entrada que la producen. Si una entrada x_j conserva rango ancho con la salida acotada $\rightarrow$ robusta frente a x_j ; si se contrae a un rango milimétrico $\rightarrow$ factor crítico y sensible.

```
import plotly.express as px
fig = px.parallel_coordinates(df, color="sharpe",
                             color_continuous_scale=px.colors.diverging.Tealrose,
                             color_continuous_midpoint=df["sharpe"].median())
fig.show()
```

```
import hiplot as hip
hip.Experiment.from_dataframe(df).display() # hasta ~500k filas en Jupyter
```

Pair plot (`seaborn.pairplot(..., corner=True)`) para interacciones por pares y fronteras físicas.

Contornos 2D / Design Space: congela las variables poco influyentes en su mediana y evalúa el sustituto sobre una malla fina de las dos más críticas.

CRÍTICO · EVITA RADAR Y T-SNE/UMAP

Radar: el área del polígono se distorsiona cuadráticamente y depende del orden de los ejes. **t-SNE/UMAP:** destruyen distancias globales; la proximidad en el mapa $\neq$ proximidad en el espacio físico, lo que invalida todo análisis de perturbación ϵ . Para robustez prefiere PCA (lineal e invertible) — pero recuerda: PCA es visualización honesta, *no* es el método de robustez.

3. Modelado sustituto y validación HONESTA

Ajusta un sustituto $\hat{y}(x)$: Gaussian Process (suave/probabilístico) o gradient boosting (rugoso). Pero la parte que mata proyectos no es el ajuste — es la **validación**.

INTEGRADO · EL HUECO MÁS GRAVE DE LAS VERSIONES ANTERIORES

Un GP o Random Forest puede dar un RMSE precioso en **k-fold aleatorio** y ser basura justo en las zonas poco muestreadas — que son donde buscas robustez. El k-fold aleatorio mezcla puntos vecinos entre train y test, así que el modelo se "auto-rescata" y te miente. **Nunca confíes en validación aleatoria para esto.**

Usa:

- **Validación por bloques/regiones del espacio:** deja fuera regiones enteras del espacio de entrada y mira si el sustituto las predice. Expone la incapacidad de extrapolar.
- **Walk-forward (si hay eje temporal):** entrena en pasado, valida en futuro, avanza la ventana. Obligatorio en trading.
- **Gráfico de paridad por región:** predicho vs. observado, coloreado por densidad de datos local. Si los errores explotan en zonas ralas, tu mapa de robustez ahí es ficción.

```
from sklearn.model_selection import TimeSeriesSplit # walk-forward
tscv = TimeSeriesSplit(n_splits=5)
for tr, te in tscv.split(X):
    model.fit(X[tr], y[tr])
    # evaluar en X[te] – SIEMPRE futuro respecto a X[tr]

# Validación por bloques espaciales: agrupa por celdas de una rejilla
# y deja fuera celdas completas (GroupKFold sobre etiquetas de celda)
```

TRADING · GP NO EXTRAPOLA, Y ESO ES PELIGROSO CON RÉGIMENES NUEVOS

Un GP revierte a su media (o dispara su σ) fuera de la nube de datos. Si entrenaste solo en un régimen alcista de baja volatilidad, el sustituto no "sabe" nada de un crash y te dará predicciones engañosamente suaves o una σ epistémica que confundirás con riesgo. Valida explícitamente que tu sustituto cubre los regímenes que te importan, o segmentalo por régimen.

4. Cuando el sistema NO es suave: discontinuidades y regímenes

INTEGRADO · MATIZ CLAVE QUE FALTABA

Muchos sistemas reales — y casi todos los de trading — **no son suaves**: thresholds, stop-loss/take-profit, reglas discretas, cambios de régimen. Un GP con kernel Matern *interpola suavemente sobre un escalón que no existe* y te miente. En estos casos un kernel suave es activamente dañino.

Alternativas honestas: **quantile regression forests** y **NGBoost** (dan incertidumbre sin asumir suavidad), ensembles de árboles, o **segmentar el modelo por régimen** y ajustar un sustituto por régimen. La incertidumbre la obtienes de la dispersión del ensemble, no de un kernel.

```
# Incertidumbre sin asumir suavidad (quantile forest)
from sklearn.ensemble import RandomForestRegressor
import numpy as np
rf = RandomForestRegressor(n_estimators=500).fit(X, y)
# dispersión entre árboles ~ incertidumbre local
preds = np.stack([t.predict(X_grid) for t in rf.estimators_])
mu, sd = preds.mean(0), preds.std(0)
```

5. Sensibilidad global: qué variables mandan

Morris para cribado barato ($N=20-50$): descarta variables de efecto despreciable y congélalas en su mediana, reduciendo de ~ 10 a las 4–5 dominantes. Luego **Sobol** para descomponer varianza: S_1 = efecto propio, S_{-T} = incluye interacciones. Si $S_{-T} \gg S_1$, hay acoplamiento fuerte y esa variable no se ajusta de forma aislada.

```
from SALib import ProblemSpec
sp = ProblemSpec({"names": names,
                  "bounds": bounds,
                  "outputs": ["Y"]})
(sp.sample_morris(50).evaluate(lambda X: surrogate.predict(X)).analyze_morris()) # screening
(sp.sample_sobol(1024).evaluate(lambda X: surrogate.predict(X)).analyze_sobol()) # varianza
```

CRÍTICO · DATOS CORRELACIONADOS ROMPEN SOBOL; POCOS DATOS TAMBIÉN

Sobol asume **entradas independientes**. Con datos pasivos correlacionados (típico fuera de un DOE), los índices se vuelven engañosos — usa *Shapley effects*, que sí manejan correlación. Y si tienes **<50 filas**, Sobol no es válido (necesita $N \cdot (2k+2)$ evaluaciones): quédate con Morris y con el error de validación walk-forward como medida.

6. Puntuación de robustez por Monte Carlo

Evaluar un candidato no es calcular $\hat{y}(x^*)$, sino inyectar perturbaciones que simulen el ruido real y medir cuánto se mueve la salida:

```
import numpy as np
def robustness_score(x_star, surrogate, sigma_eps, n=2000, k=2):
    eps = np.random.normal(0, sigma_eps, size=(n, x_star.size))
    Y = surrogate.predict(x_star + eps)
    media, desv = Y.mean(), Y.std()
    objetivo_robusto = media + k * desv # Beyer & Sendhoff, 2007
    p_in_spec = (Y > umbral_min).mean() # prob. de cumplir especificación
    return media, desv, objetivo_robusto, p_in_spec
```

INTEGRADO · LO QUE NADIE (NI CHATGPT NI YO) HABÍA ENFATIZADO: CÓMO FIJAR SIGMA_EPS

Todo el método depende de este número. Si lo eliges mal, el análisis es teatro. `sigma_eps` debe venir de la **variabilidad real**, no de un valor cómodo:

- **Tolerancia de proceso / instrumento:** la desviación con que de verdad puedes fijar cada variable.
- **Trading:** la incertidumbre real de tus parámetros — slippage, latencia, ejecución, y sobre todo **la variación entre re-estimaciones del parámetro en distintas ventanas**. Si tu "lookback óptimo" salta entre 18 y 34 según el periodo, tu `sigma_eps` en esa variable es enorme, no cero.
- **Regla práctica:** mide la dispersión histórica de cada parámetro óptimo entre sub-muestras y úsala como σ . Es honesto y autoinformado por los datos.

CRÍTICO · EPISTÉMICA ≠ ALEATORIA

`gpr.predict(X, return_std=True)` devuelve incertidumbre **epistémica** (cuánto duda el modelo). **No** es la varianza del sistema ante ruido de entrada (**aleatoria**). Para robustez operativa es obligatorio el bucle Monte Carlo de arriba.

7. La maldición de la dimensionalidad y el muestreo eficiente

En 8–10D los grids regulares explotan combinatoriamente y el Monte Carlo ingenuo se vuelve carísimo. Usa muestreo que llene el espacio con menos puntos:

Técnica	Cuándo	Python
Latin Hypercube (LHS)	Cobertura uniforme con pocos puntos	<code>scipy.stats.qmc.LatinHypercube</code>
Quasi-Monte Carlo (Sobol/ Halton)	Convergencia más rápida que MC puro	<code>scipy.stats.qmc.Sobol</code>
Active learning	Si puedes generar datos nuevos*	modAL, BoTorch
Optimización bayesiana adaptativa	Si puedes generar datos nuevos*	Optuna, scikit-optimize

* Active learning y BO adaptativa requieren que puedas *ejecutar nuevos experimentos/backtests* a demanda. Con un dataset fijo y pasivo no aplican; ahí LHS/QMC sobre el sustituto es lo correcto.

8. Robustez multiobjetivo (en trading, casi obligatorio)

INTEGRADO · REEVALUADO AL ALZA POR EL CONTEXTO DE TRADING

Rara vez optimizas un solo objetivo. En trading quieres a la vez maximizar retorno, minimizar drawdown, minimizar sensibilidad a parámetros y minimizar costos. No hay un ganador único: hay un **frente de Pareto** de soluciones no dominadas. La elección entre ellas es tuya, pero la robustez puede entrar como un objetivo más (minimizar σ del Monte Carlo) o como restricción ($\sigma < \tau$).

Herramientas: **Optuna** (multiobjetivo, `directions=["maximize","minimize"]`), NSGA-II vía `pymoo`, e hipervolumen como métrica de calidad del frente. Visualiza el frente con scatter coloreado o coordenadas paralelas sobre los objetivos.

9. Trampas específicas de trading / series temporales

TRADING · ESTO HUNDE MÁS ESTRATEGIAS QUE CUALQUIER PROBLEMA DE VARIANZA

- **Lookahead bias (fuga temporal)**: usar información del futuro al construir features o al normalizar (p. ej., escalar con la media de todo el dataset, incluido el test). Calcula z-scores y estadísticas *solo con datos pasados, rolling*.
- **Sobreajuste de backtest / p-hacking**: probar mil combinaciones y quedarte con la mejor garantiza un ganador por azar. Corrige por múltiples pruebas (*Deflated Sharpe Ratio*, López de Prado) y reserva un *hold-out* que no toques jamás hasta el final.
- **Drift / no estacionariedad**: el sistema cambia con el tiempo. Una región robusta en 2019–2021 puede no serlo en 2024. Re-valida walk-forward y vigila la estabilidad del parámetro óptimo entre ventanas (esa inestabilidad es tu `sigma_eps`, ver §6).
- **Purga y embargo**: al validar con datos solapados (features con ventana), purga las observaciones cercanas a la frontera train/test y aplica un *embargo* temporal para evitar fuga por autocorrelación.

10. Taxonomía de incertidumbre (marco mental)

Útil para pensar, aunque no toda sea accionable hoy: **de medición** (ruido del instrumento → propágala con Monte Carlo), **estructural** (el modelo es incompleto), **del surrogate** (error de ajuste → validación honesta), **de extrapolación** (fuera de la nube de datos → la más peligrosa, ver §3), y **drift temporal** (el proceso cambia → walk-forward y monitoreo). En trading las dos que más dañan son extrapolación y drift, no la de medición.

11. Tabla de herramientas

Tarea	Librería	Nota
Exploración visual interactiva	HiPlot / Plotly Express	Brushing multidimensional
Interacciones por pares	seaborn (pairplot)	Fronteras físicas en 2D
Cribado de variables	SALib (Morris)	Barato, N=20–50
Sensibilidad global	SALib (Sobol) / Shapley	Shapley si hay correlación
Sustituto suave	scikit-learn (GP + WhiteKernel)	Solo si el sistema es suave
Sustituto rugoso / discontinuo	lightgbm, NGBoost, quantile forest	Thresholds, regímenes, stops
Validación temporal	sklearn TimeSeriesSplit / mlfinlab	Walk-forward, purga, embargo

Tarea	Librería	Nota
Muestreo eficiente	scipy.stats.qmc (LHS, Sobol)	Alta dimensión
Multiobjetivo	Optuna, pymoo (NSGA-II)	Frentes de Pareto

12. Flujo de trabajo secuencial

1. **Limpieza:** pairplot + correlación de Spearman; elimina datos imposibles/duplicados y detecta variables redundantes.
2. **Sustituto base:** Random Forest rápido; revisa importancias como validación preliminar.
3. **Validación honesta (no negociable):** walk-forward y/o por bloques espaciales. Si el sustituto no extrapola, no sigas hasta arreglarlo.
4. **Cribado (Morris):** congela variables despreciables; reduce a las 4–5 dominantes.
5. **Modelo de alta fidelidad:** GP+WhiteKernel si es suave; quantile forest/NGBoost si hay discontinuidades o regímenes. Valida con paridad por región.
6. **Calibra sigma_eps** desde tolerancias reales o desde la dispersión del parámetro óptimo entre ventanas. *No lo inventes.*
7. **Monte Carlo de robustez** sobre una malla LHS/QMC del espacio reducido; calcula μ , σ , $P(\text{en-especificación})$.
8. **Mapeo Design Space (QbD):** contornos 2D de $P(\text{en-especificación}) > 0.95$ sobre las dos variables de mayor S_T , con pequeños múltiplos para una tercera.
9. **Multiobjetivo (si aplica):** frente de Pareto retorno / drawdown / σ ; elige sobre el frente.
10. **Confirmación:** en trading, prueba forward en datos jamás tocados (paper trading o hold-out final). En sistemas físicos, 3–5 corridas reales dentro de la región robusta.